2. НАСТРОЙКА РАБОЧЕГО ОКРУЖЕНИЯ

В ходе выполнения индивидуального задания в рамках учебной практики был сформирован технологический стек, включающий язык программирования Python, специализированную среду разработки PyCharm и библиотеку для создания графических интерфейсов PyQt5.

2. Настройка рабочего окружения

В ходе прохождения учебной практики использовался язык программирования Python, среда разработки (IDE) PyCharm

Python – это высокоуровневый язык программирования, с помощью которого создают сайты, разрабатывают приложения, автоматизируют процессы анализа или визуализации данных. Python не был разработан для конкретных целей, поэтому подходит как для создания алгоритма рекомендаций видеосервиса, так и для разработки программного обеспечения и других целей. Согласно индексу TIOBE, Python стабильно входит в тройку самых популярных языков программирования, а по данным опроса Stack Overflow 2024 года, этот язык используют 51% разработчиков [1].

Ключевые технические характеристики Python:

- Объектно-ориентированный подход: в основе языка лежат концепции объектов и классов, что позволяет проектировать сложные системы с четкой иерархией и повторно используемым кодом.
- Высокая читабельность: синтаксис языка ориентирован на минимизацию объема кода. Использование обязательных отступов для выделения логических блоков исключает избыточность и облегчает процесс командной разработки и ревью.
- Интерпретируемость и кроссплатформенность: код исполняется последовательно с помощью программы-интерпретатора, что позволяет запускать приложения на различных операционных системах без необходимости перекомпиляции под каждую платформу.

Преимущества Python:

- хорошо подходит для новичков;
- простой минималистичный синтаксис: код легко писать, читать и поддерживать;
- большая стандартная библиотека и много дополнительных библиотек;
- большой выбор фреймворков;
- поддерживает объектно-ориентированное программирование и другие парадигмы;
- кроссплатформенность и поддержка почти всех современных систем.

Недостатки Python:

- низкая скорость;
- из-за динамической типизации выше вероятность ошибки при запуске, нужно больше тестов;
- не подходит для работы с памятью на низком уровне.

2.2 Выбор среды разработки (IDE)

Для написания и отладки программного кода была выбрана интегрированная среда разработки PyCharm от компании JetBrains. Она представляет собой профессиональный инструмент, оптимизированный специально для работы с Python-проектами.

Функциональные возможности и преимущества PyCharm:

1. Автоматизация контроля качества: встроенные инструменты статического анализа проверяют код на соответствие стандарту PEP 8, выявляя потенциальные ошибки и стилистические недочеты в режиме реального времени.
2. Интеллектуальная поддержка: контекстное автодополнение кода, глубокая навигация по структуре проекта и инструменты безопасного рефакторинга значительно ускоряют процесс написания модулей.

3. Интеграция с профессиональным инструментарием: наличие встроенного отладчика, поддержка систем управления пакетами и тесная интеграция с системой контроля версий Git.

4. Виртуальные окружения: возможность создания изолированных сред для каждого проекта, что исключает конфликты зависимостей между различными библиотеками.

К недостаткам среды можно отнести высокую требовательность к аппаратным ресурсам (объему оперативной памяти) и относительную сложность первичной настройки интерфейса для начинающих специалистов.

Рисунок 1 – Среда разработки PyCharm

2.3 Выбор библиотек и фреймворков

Особое значение при проектировании десктопного приложения имеет выбор инструментария для создания графического интерфейса пользователя (GUI). После анализа доступных решений (Tkinter, PySide, CustomTkinter) выбор был сделан в пользу библиотеки PyQt5.

PyQt5 представляет собой набор Python-связок для кроссплатформенного фреймворка Qt. Выбор данной библиотеки обоснован следующими факторами:

- PyQt5 предоставляет обширный инструментарий для реализации сложных форм ввода, динамических таблиц, графиков и вкладок;

– использование механизмов гибкой компоновки позволяет создавать адаптивные интерфейсы, корректно отображающиеся при изменении размеров окна;

– механизм сигналов и слотов обеспечивает эффективное взаимодействие между элементами интерфейса и вычислительной логикой;

– встроенные классы (QIntValidator, QDoubleValidator) позволяют ограничить ввод некорректных данных на уровне интерфейса, повышая отказоустойчивость программы.

Такой технологический стек оптимально соответствует целям проекта: он обеспечивает минимальное время реализации и высокую поддерживаемость кода, что делает его наиболее подходящим выбором для создания десктопного приложения.

Скриншоты работы в выбранной среде приведены на рисунках 2-4.

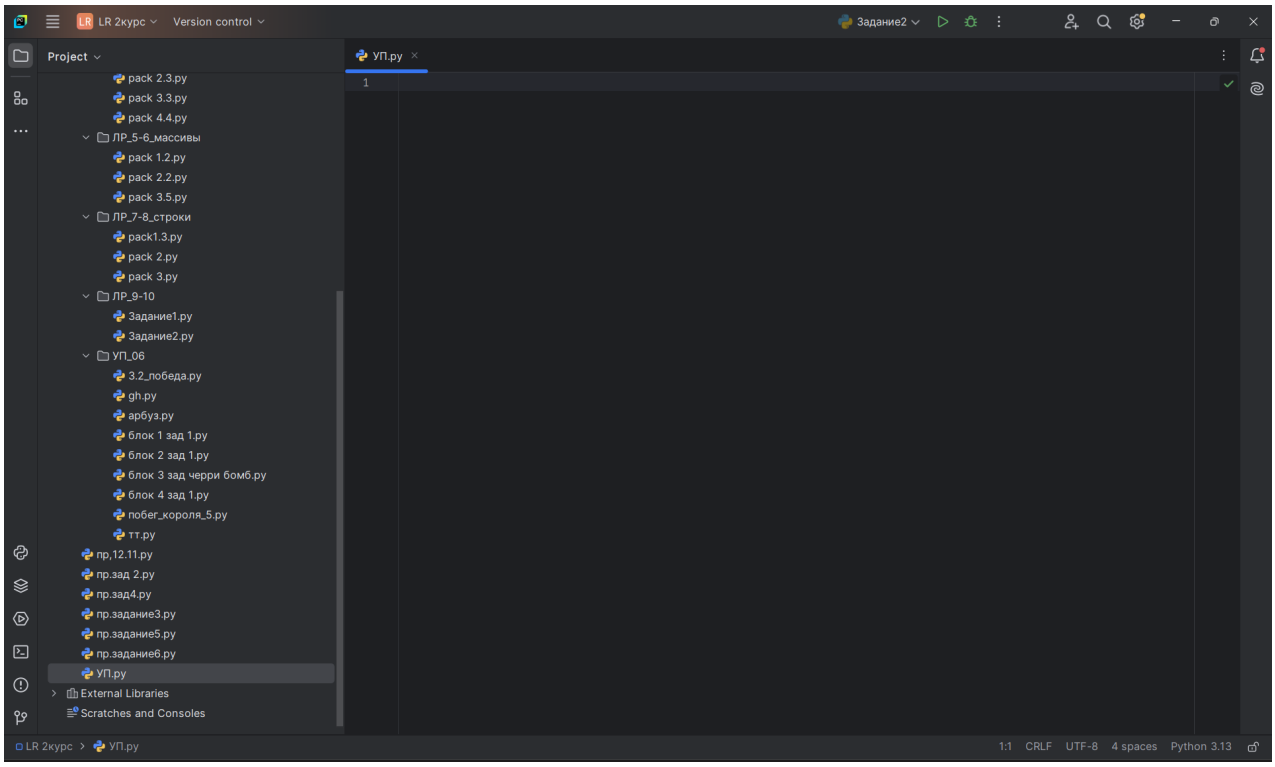


Рисунок 2 – Интерфейс PyCharm

Для системы контроля версий в соответствии с заданием использована система GitHub. Ссылка на аккаунт в GitHub: kyomii09

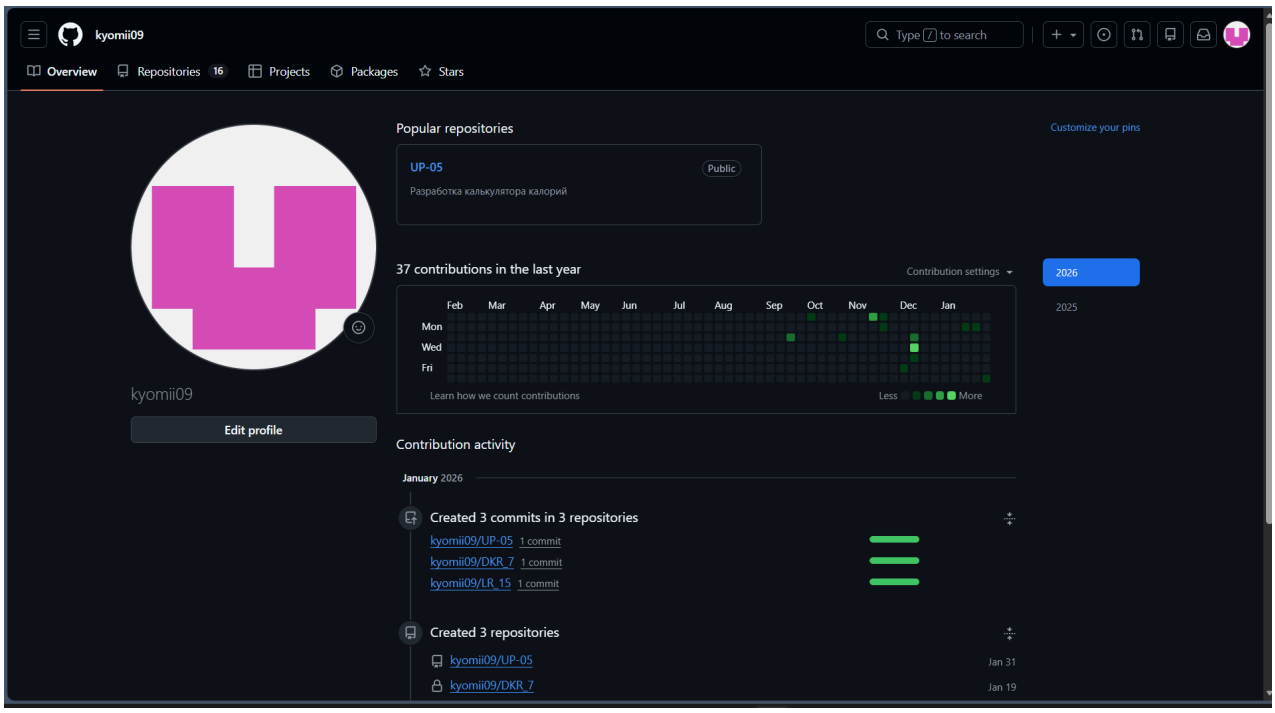


Рисунок 3 – Аккаунт на GitHub

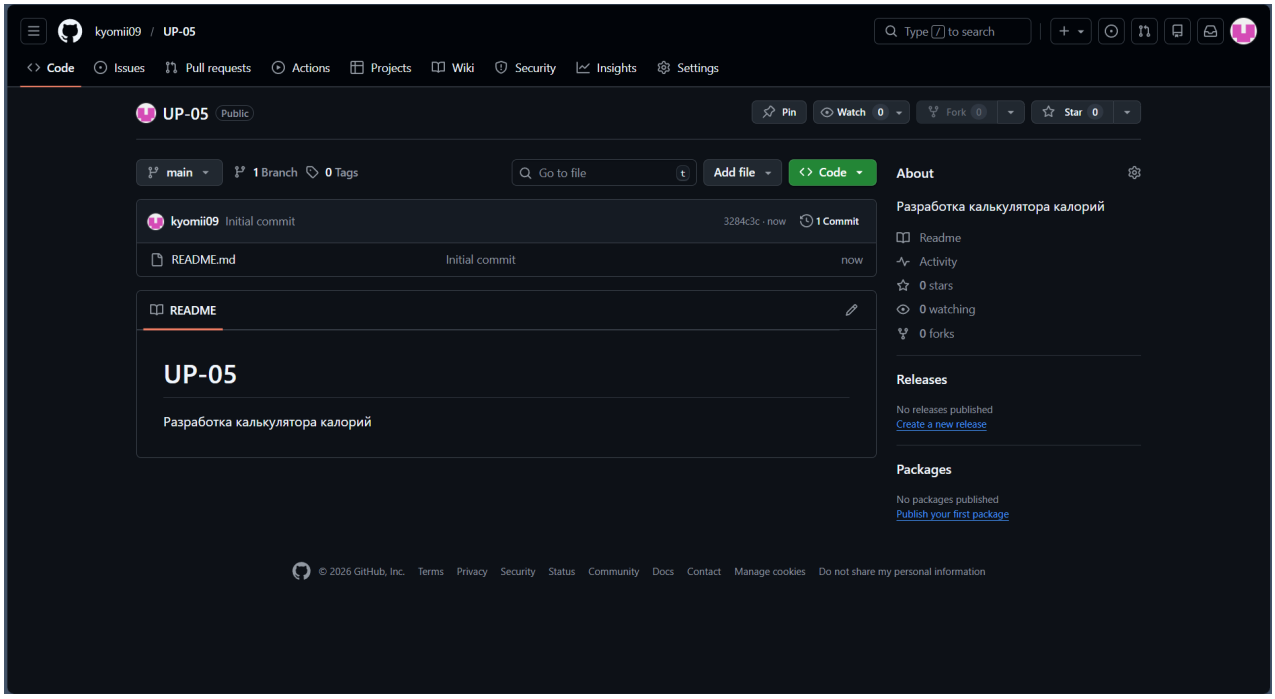


Рисунок 4 – Репозиторий проекта на GitHub

4. ОПИСАНИЕ ВЫПОЛНЕНИЯ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

4.1 Анализ предметной области

Процесс функционирования человеческого организма носит непрерывный характер и требует постоянных энергетических затрат, которые не прекращаются даже в состояниях глубокого сна, физического покоя или психоэмоционального отдыха. Энергия, выраженная в килокалориях (ккал), является необходимым ресурсом для поддержания витальных функций: стабильной работы сердечно-сосудистой системы, осуществления дыхательной деятельности, функционирования коры головного мозга, процессов фильтрации в почках, а также метаболической активности печени и органов пищеварения. Постоянный расход энергии в условиях покоя обуславливает физиологическую потребность в регулярном поступлении и правильном распределении питательных веществ.

Фундаментальным аспектом управления массой тела и превентивной медицины является контроль суточного энергопотребления. Энергетический баланс – это разница между энергией, полученной из пищи, и энергией, затраченной в ходе жизнедеятельности. Данное соотношение определяет динамику веса:

- избыток калорий ведет к депонированию энергии в виде жировых запасов;
- недостаток калорий инициирует мобилизацию внутренних резервов организма;
- состояние баланса, обеспечивающее поддержание стабильных антропометрических показателей.

Суточные энергозатраты человека (TDEE – Total Daily Energy Expenditure) представляют собой комплексную величину, состоящую из трех ключевых компонентов:

1. Базальный метаболизм (BMR): базовый расход энергии (60–70% от общего объема), необходимый для поддержания жизни в состоянии абсолютного покоя. Он детерминирован генетическими факторами, полом, возрастом и безжировой массой тела.

2. Термический эффект пищи (ТЭП): затраты на переваривание и усвоение нутриентов (10–15% рациона). Важно отметить, что белковые структуры требуют на метаболизацию до 30% своей калорийности, что делает их приоритетными при составлении диетологических планов.

3. Физическая активность: наиболее вариабельный фактор (15–50%), охватывающий как тренировочный процесс, так и повседневную двигательную активность. Для его оценки применяются коэффициенты физической активности (КФА) в диапазоне от 1.2 до 1.9.

Для автоматизации процессов индивидуального планирования питания в программном обеспечении реализована поддержка четырех научно верифицированных математических моделей:

- Алгоритм Миффлина – Сан Жеора: признанный современный стандарт для взрослого населения.
- Формула Шофилда: специализированный метод для детей и подростков (до 18 лет), учитывающий динамические процессы роста.
- Метод Кетча – МакАрдла: высокоточный алгоритм для людей с развитой мышечной массой, базирующийся на проценте подкожного жира.
- Стандарт ВОЗ: клиническая модель Всемирной организации здравоохранения для универсальной оценки потребностей организма.

В разрабатываемом ПО реализован принцип безопасной коррекции рациона. В отличие от радикальных методик, программа применяет фиксированную корректировку в размере 500 ккал от нормы TDEE. Данный подход признан «золотым стандартом» диетологии, так как обеспечивает стабильную динамику изменения веса (около 0.5 кг в неделю) без введения организма в состояние метаболического стресса. Дополнительно система осуществляет расчет макронутриентов в сбалансированной пропорции: белки – 30%, жиры – 30%, углеводы – 40%.

4.2 Обоснование актуальности разработки на основе рыночных данных

Анализ востребованности подобных инструментов в русскоязычном сегменте интернета, проведенный на основе данных сервиса «Яндекс.Вордстат» за период декабрь 2025 – январь 2026 года, подтверждает высокую актуальность задачи. Статистические данные указывают на следующие показатели:

- Количество прямых поисковых запросов «калькулятор калорий» составило 114 294 единицы.
- Количество связанных информационных запросов «сколько ккал» достигло 238 740 единиц.

Такая высокая частотность свидетельствует о сформированной потребности широкой аудитории в инструментах для количественного анализа рациона. Несмотря на наличие онлайн-решений, существует проблема отсутствия надежных десктопных инструментов, обеспечивающих конфиденциальность данных и высокую скорость работы без доступа к сети Интернет.

4.3 Анализ проблемы

Глобальная заболеваемость, связанная с избыточной массой тела и ожирением, демонстрирует устойчивую тенденцию к росту, охватывая как взрослое население, так и детей. Попытки самостоятельной коррекции массы тела зачастую осуществляются без опоры на научно обоснованные методы, что приводит к неэффективности вмешательств и потенциальным рискам для здоровья. Распространённые ошибки включают:

- применение несбалансированных режимов питания;
- игнорирование индивидуальных физиологических параметров;
- отсутствие системного подхода к контролю энергетического баланса.

Основной причиной указанных проблем является отсутствие доступного программного инструмента, обеспечивающего практическое применение

научных знаний в области энергетического обмена. Несмотря на общедоступность информации о роли калорийного баланса, его реализация в повседневной практике затруднена следующими факторами:

- сложность расчёта индивидуальной нормы энергозатрат с учётом антропометрических данных;
- трудоёмкость ручного учёта потреблённой энергии.

Для устранения данных ограничений разработано десктопное приложение «Калькулятор калорий», которое обеспечивает автоматизацию ключевых этапов управления энергетическим балансом:

- расчёт базального метаболизма с использованием формулы Миффлина – Сан Жеора;
- коррекция на уровень физической активности через стандартизированную шкалу коэффициентов;
- определение общей суточной энергозатратности;
- ведение учёта потреблённой энергии с визуализацией остатка калорий в реальном времени.

Приложение устраняет разрыв между теоретическими знаниями о регуляции энергетического баланса и их практическим применением, предоставляя пользователю точный, персонализированный и удобный инструмент для ежедневного контроля потребления энергии. Это позволяет преодолеть основные барьеры самостоятельного управления массой тела – субъективность оценок, отсутствие индивидуального расчёта нормы калорий и недостаток обратной связи – и тем самым способствует достижению устойчивых, безопасных и физиологически обоснованных результатов.

4.4 Обзор аналогов

Первым рассматриваемым аналогом является «Счётчик калорий онлайн, расход калорий» на справочном портале Calc.ru. Данный сервис представляет собой специализированный веб-инструмент для расчёта энергозатрат при различных видах физической активности, что отличает его от калькуляторов суточной нормы (интерфейс представлен на рисунке 1).

Калькулятор позволяет выбрать конкретный вид активности, указать продолжительность выполнения в минутах и вес пользователя. На основе этих данных система в реальном времени вычисляет и отображает общее количество израсходованных килокалорий.

Достоинства данного калькулятора:

- узкая специализация и простота – решение одной задачи (расчёт расхода калорий) через минималистичный интерфейс;
- мгновенный результат – динамический расчёт с выводом итога после ввода данных;

Основные недостатки калькулятора:

- ограниченный, разовый функционал – отсутствие истории, сравнения активностей или дневника тренировок;
- архитектурная ограниченность веб-сервиса – потеря всех данных после обновления или закрытия страницы;
- наличие агрессивной рекламы – перегруженность интерфейса баннерами, ухудшающая пользовательский опыт;

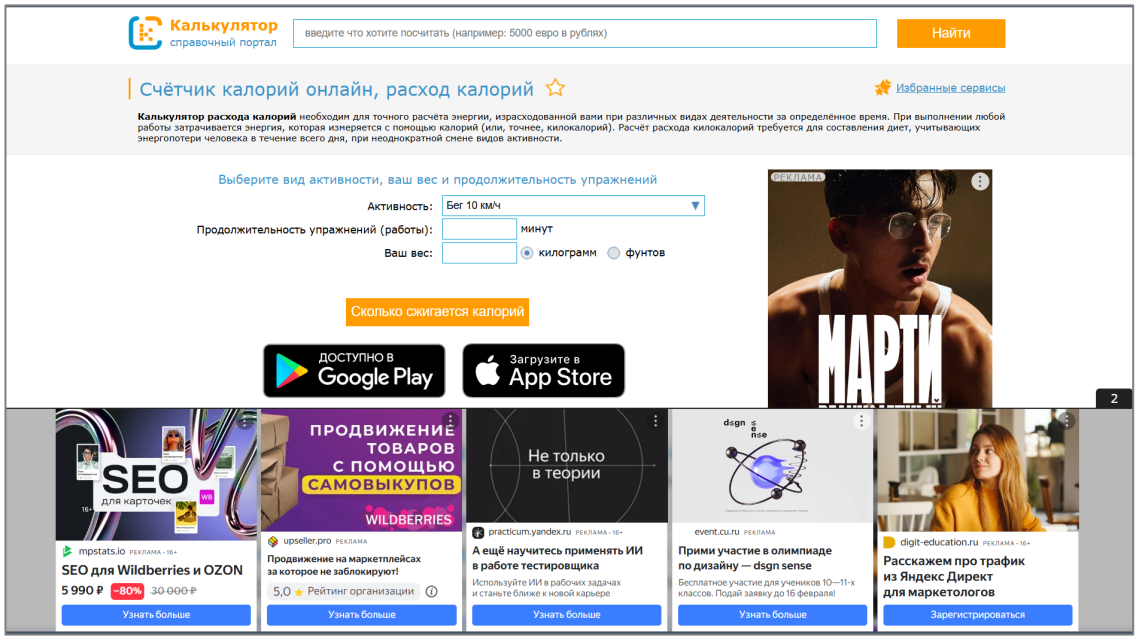


Рисунок 5 – Интерфейс онлайн-калькулятора расхода калорий на портале Calc.ru

Вторым рассматриваемым аналогом является калькулятор суточной нормы калорий, размещённый на авторитетном медицинском портале RLSNET.RU. Данный сервис ориентирован на расчёт энергозатрат по классической формуле Харриса–Бенедикта и позиционируется как инструмент для профессионального и полупрофессионального использования (интерфейс представлен на рисунке 2).

Достоинства:

- высокая достоверность источника – портал RLSNET является признанным справочным ресурсом в области фармакологии и клинической медицины;
- прозрачность расчёта – чётко указано, какая формула используется и как применяется коэффициент активности;
- простота и скорость получения результата без необходимости регистрации или перехода по рекламным баннерам.

Основным недостатком данного решения является его функциональная ограниченность. Калькулятор представляет собой одноразовый расчётный инструмент, не предусматривающий ни учёта фактического потребления пищи, ни сохранения пользовательских данных. После закрытия страницы вся информация теряется, а повторное использование требует повторного ввода всех параметров. Более того, сервис не поддерживает выбор современных формул, что снижает точность расчётов для большинства взрослых людей, особенно с избыточной массой тела. Отсутствие возможности корректировать цель (похудение, набор, поддержание) и отсутствие обратной связи в течение дня делают этот инструмент непригодным для долгосрочного контроля питания и управления весом.

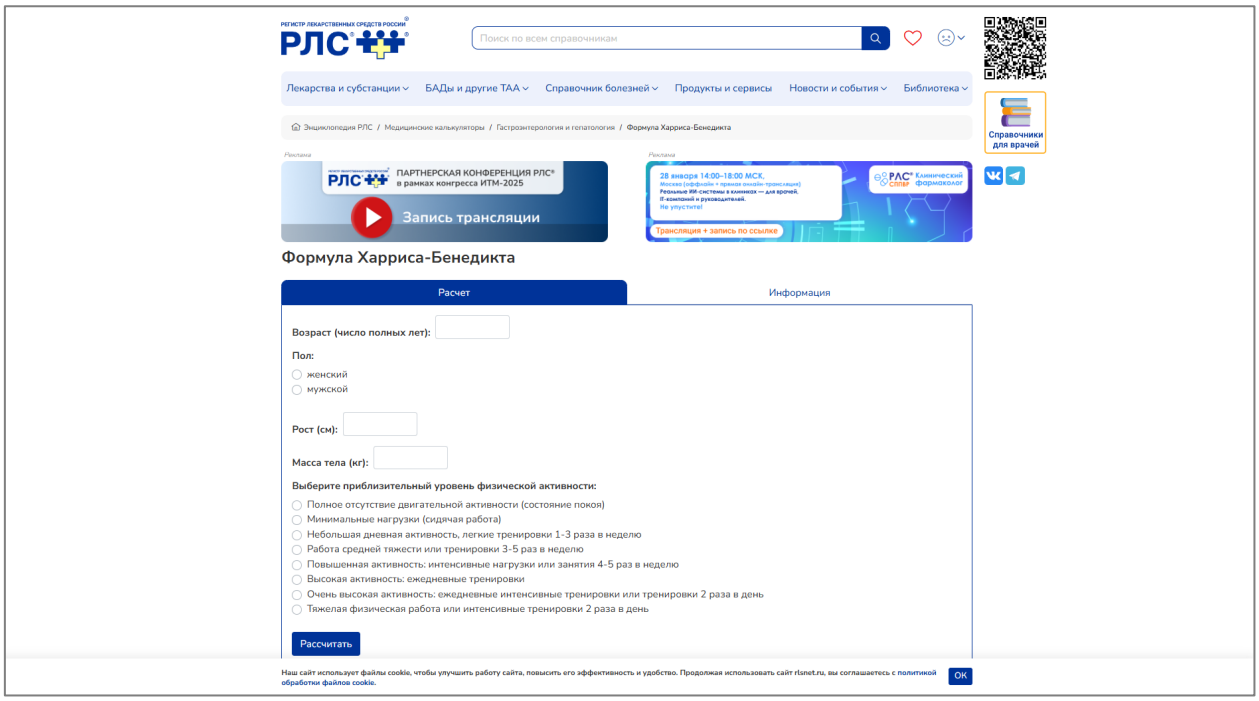


Рисунок 6 – Интерфейс онлайн-калькулятора расхода калорий на портале RLSNET.RU

Третьим рассматриваемым аналогом является калькулятор суточной нормы калорий, DoctorUshakov.com. Данный сервис позиционируется как простой и доступный инструмент для расчёта энергетических потребностей с учётом цели пользователя: поддержание веса, похудение или набор массы (интерфейс представлен на рисунке 3).

Достоинства калькулятора DoctorUshakov.com:

– ориентация на конечную цель пользователя (похудение, набор, поддержание), что делает результат более практичным;

– автоматический расчёт не только калорий, но и рекомендуемого количества белков, жиров и углеводов (БЖУ);

– наличие пояснений под каждым пунктом интерфейса, что снижает порог входа для новичков;

– адаптация под российскую аудиторию – используются привычные единицы измерения и термины.

Основным недостатком данного решения является его статичная и одноразовая природа. Как и большинство онлайн-калькуляторов, он не

сохраняет данные пользователя и не предоставляет функцию учёта фактического питания. После получения цифры пользователь остаётся один на один с задачей самостоятельно отслеживать, сколько калорий он уже съел за день. Кроме того, методика расчёта не указана явно – неясно, какая формула используется (Харрис–Бенедикт, Миффлин–Сан Жеор или упрощённая модель), что снижает прозрачность и научную обоснованность результата. Отсутствие возможности выбрать формулу или скорректировать параметры (например, степень дефицита) ограничивает гибкость инструмента.

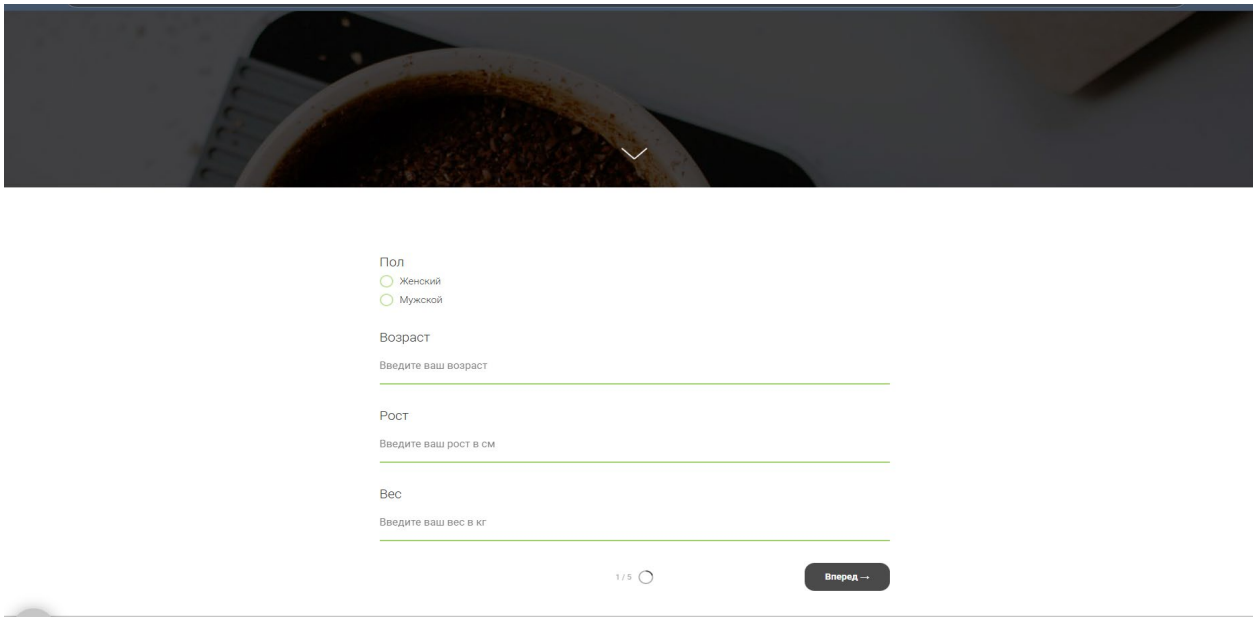


Рисунок 7 – Интерфейс онлайн-калькулятора расхода калорий на портале DoctorUshakov.com

Четвёртым рассматриваемым аналогом является онлайн-калькулятор, размещённый на сайте фитнес-тренера MaslovaFit.com. Данный сервис позиционируется как простой и быстрый инструмент для расчёта суточной нормы калорий и макронутриентов под конкретную цель: похудение, набор мышечной массы или поддержание веса (интерфейс представлен на рисунке 4).

Достоинства калькулятора MaslovaFit.com:

– высокая скорость получения результата – всё рассчитывается за несколько секунд;

– ориентация на конечную цель пользователя с автоматической коррекцией калорийности;

– предоставление не только калорий, но и рекомендаций по БЖУ и водному балансу;

– отсутствие рекламы и сложных меню – интерфейс действительно прост.

Основным недостатком данного решения является его полная статичность и отсутствие интерактивности. Калькулятор выполняет однократный расчёт и не предлагает никаких средств для дальнейшего контроля. Кроме того, методика расчёта не раскрывается – не указано, какая формула используется (Миффлин–Сан Жеор, Харрис–Бенедикт или упрощённая модель), что снижает доверие к точности результата. Отсутствие возможности выбрать формулу особенно критично для людей с избыточным весом, которым требуется более адаптированный подход (например, формула ВОЗ).

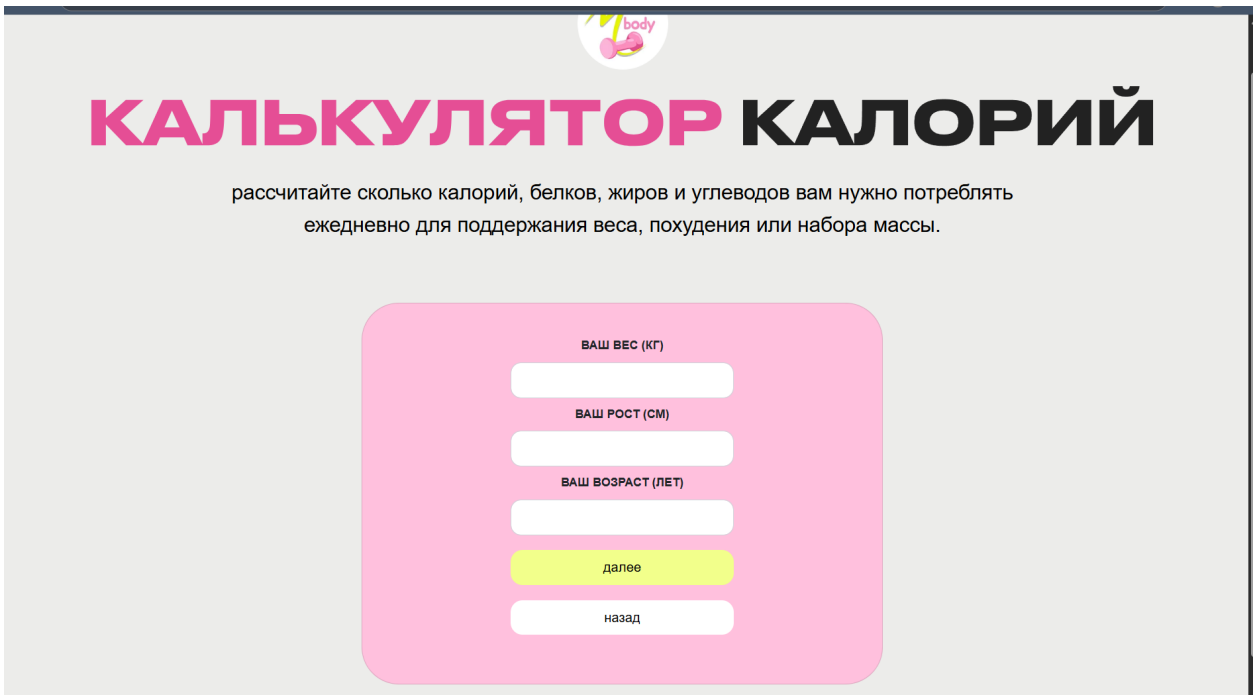


Рисунок 8 – Интерфейс онлайн-калькулятора расхода калорий на портале MaslovaFit.com

Таким образом, несмотря на различия в оформлении и дополнительных функциях, все рассмотренные аналоги объединяет один ключевой недостаток: они представляют собой одноразовые веб-инструменты, ориентированные исключительно на первоначальный расчёт нормы калорий. Ни один из них не обеспечивает сохранять персональные данные между сессиями. Кроме того, большинство сервисов не раскрывают используемые методики расчёта, ограничивают выбор формулы или предлагают жёсткие шаблоны без возможности индивидуальной настройки. В совокупности это делает их малоэффективными для повседневного применения рядовым пользователем, который ищет простой, надёжный и автономный инструмент для долгосрочного контроля энергетического баланса. Именно этот пробел и призван заполнить разрабатываемый десктопный калькулятор калорий.

4.2 Разработка структуры приложения и алгоритмов функционирования

4.2.1 Разработка алгоритмов функционирования

Алгоритм работы программы «Калькулятор калорий» представлен на рисунке 1.

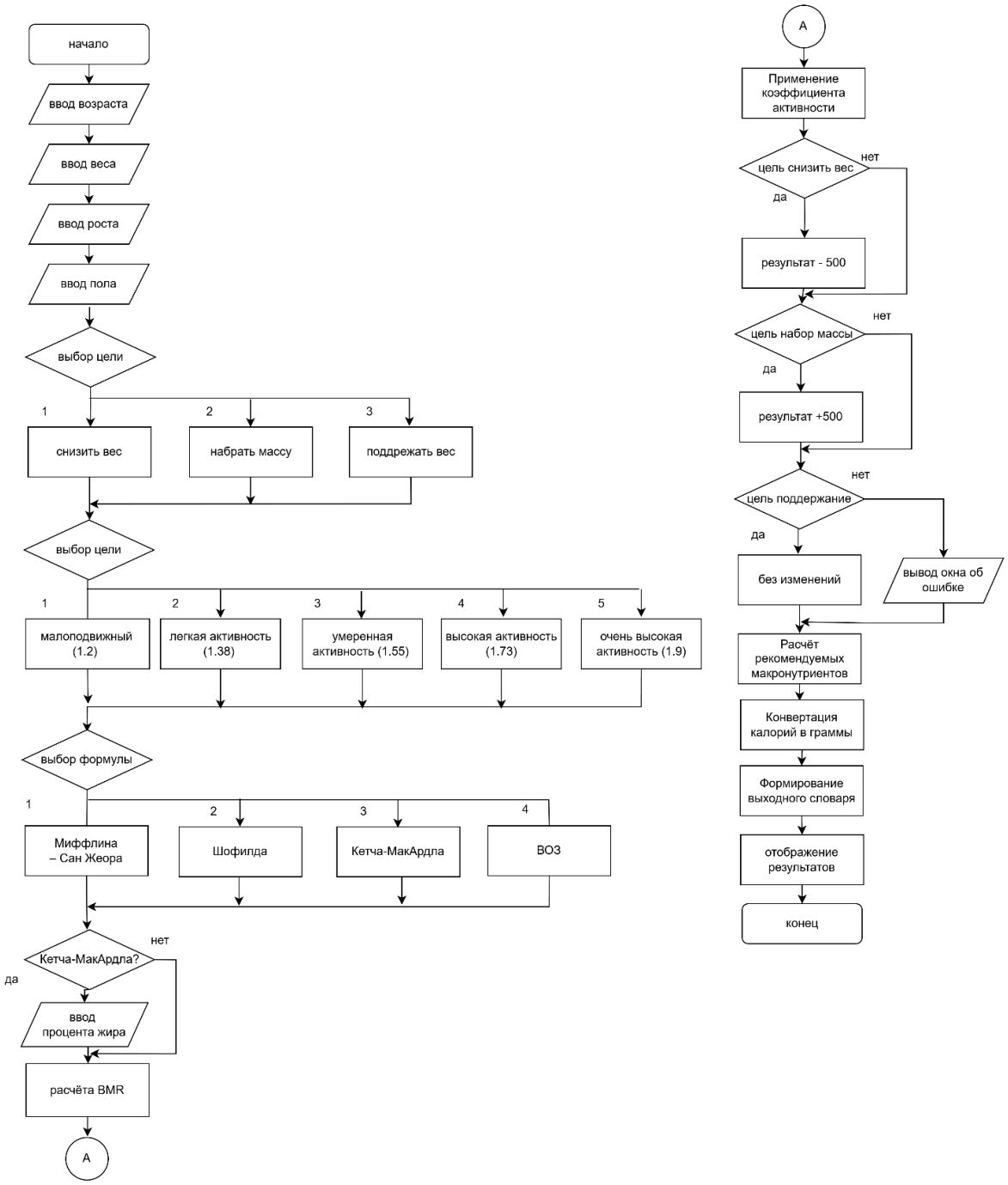


Рисунок 9 – Алгоритм функционирования программы

Алгоритм поэтапная работы программы представлен ниже.

А. При запуске исполняемого файла main.py происходит инициализация графического пользовательского интерфейса и на экране отображается главное окно калькулятора.

Б. В левой части экранной формы пользователь осуществляет ввод антропометрических данных, заполняя следующие поля:

– возраст (полных лет);

– текущая масса тела (в килограммах);

– рост (в сантиметрах);

– пол (мужской или женский);

– процент жира в организме (данное поле заполняется только в случае использования формулы Кетча – МакАрдла).

В. После внесения физических параметров производится выбор диетологической цели из предложенного перечня:

– сброс веса (дефицит калорий);

– поддержание текущего веса (баланс);

– набор массы (профицит калорий).

Г. Выбор уровня физической активности осуществляется посредством выпадающего списка, где пользователю доступно пять вариантов (от минимальной до очень высокой активности), каждому из которых соответствует математический коэффициент в диапазоне от 1.2 до 1.9.

Д. Из списка доступных вычислительных моделей выбирается одна из расчетных формул:

– Миффлина – Сан Жеора;

– Шофилда;

– Кетча – МакАрдла;

– ВОЗ (Всемирная организация здравоохранения).

Е. При активации управляющего элемента «Рассчитать» система производит комплексную валидацию данных. На данном этапе проверяется полнота заполнения полей, корректность числовых диапазонов и соответствие выбранной цели возрасту пользователя (блокировка функций изменения веса для лиц младше 18 лет). При обнаружении ошибок выводится диагностическое сообщение.

Ж. Полученный результат последовательно обрабатывается программными алгоритмами в следующем порядке:

– значение BMR умножается на выбранный коэффициент активности;

– итоговое число корректируется под выбранную цель путем вычитания 500 ккал (для похудения) или добавления 500 ккал (для набора массы);

– осуществляется финальное распределение энергии на белки (30%), жиры (30%) и углеводы (40%).

З. Рассчитанные показатели мгновенно визуализируются в правой части окна и включают в себя:

– целевую суточную калорийность;

– количество белков, жиров и углеводов в граммах;

– процентное соотношение нутриентов, отображаемое с помощью цветных индикаторов класса.

И. Пользователь может в любое время ознакомиться с теоретической базой в окне справки, изменить любые введенные данные для повторного расчета или воспользоваться функцией «Сбросить» для полной очистки всех полей ввода.

К. Завершение работы с программным комплексом осуществляется путем закрытия главного окна приложения, что приводит к прекращению всех процессов и освобождению оперативной памяти.

4.2.2 Проектирование интерфейса

В данном подразделе представлены визуальные решения и макеты экранных форм разрабатываемого приложения.

Основное рабочее пространство приложения организовано таким образом, чтобы обеспечить быстрый доступ к вводу параметров и мгновенное получение результата без переходов между множеством окон. Внешний вид макета главного окна калькулятора, включающий зоны ввода данных и панель отображения итоговых показателей, представлен на рисунке 2.

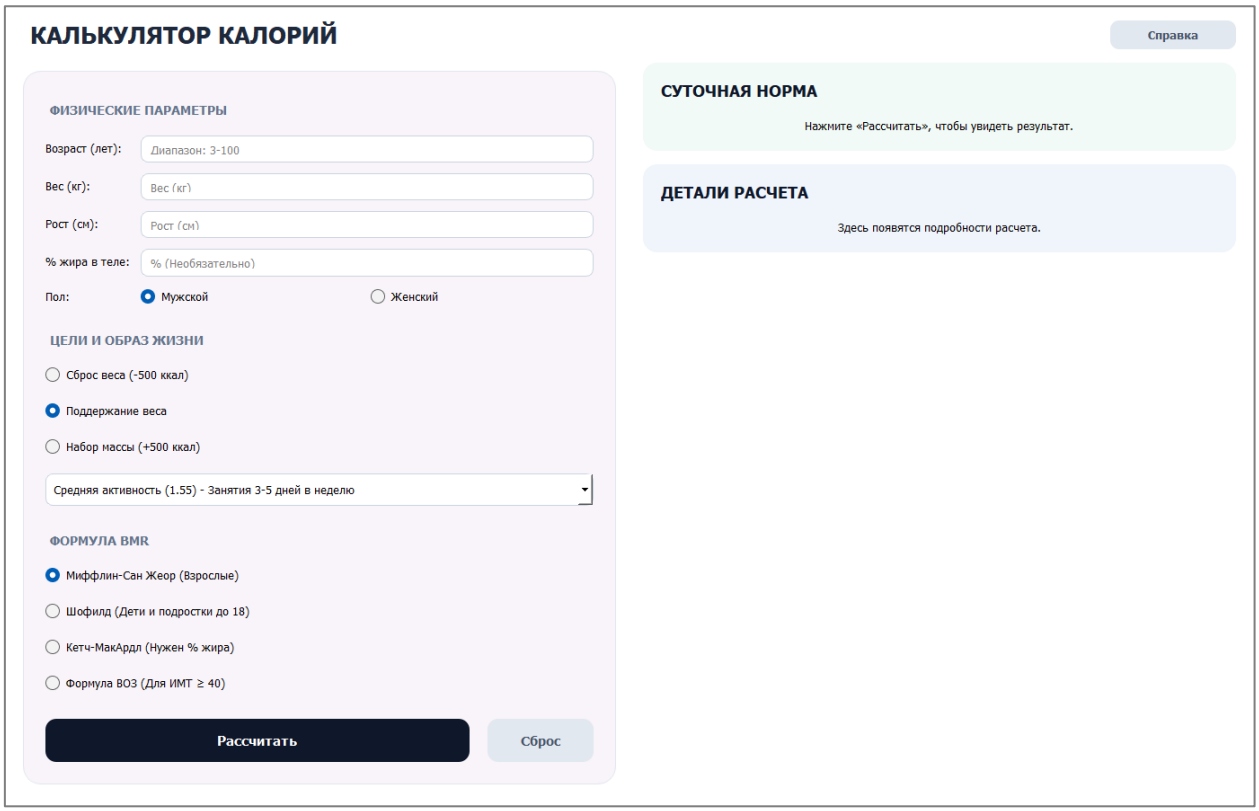


Рисунок 10 – Интерфейс программы

Для обеспечения пользователя справочной информацией о применяемых в системе математических методах было спроектировано дополнительное окно справки. Данное окно вызывается из главного интерфейса и содержит текстовые описания алгоритмов. Макет окна справки представлен на рисунке 3.

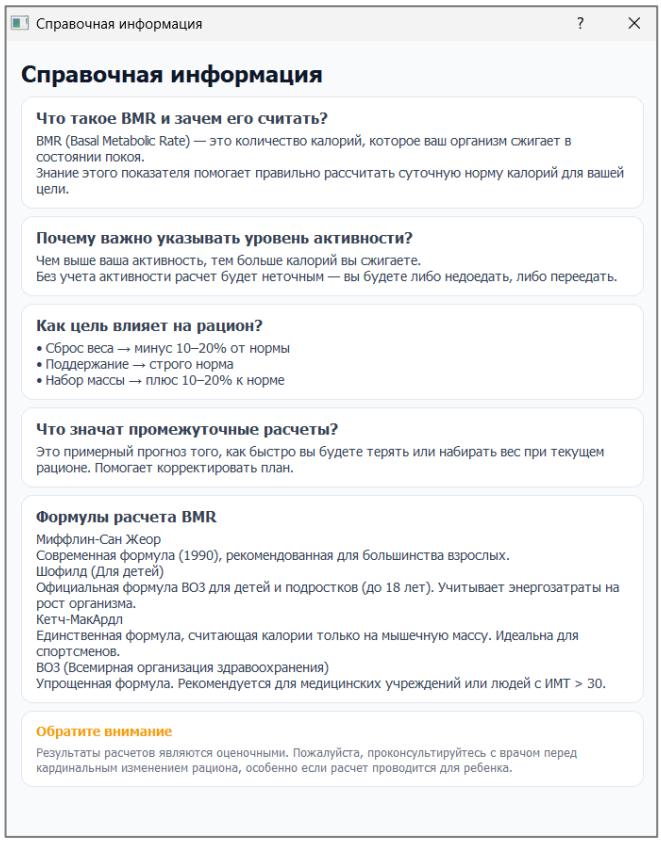


Рисунок 11 – Макет окна справки

В данном разделе рассмотрены алгоритмы работы системы и приведен прототип интерфейса. Разработанный алгоритм обеспечивает корректную последовательность расчетов: сбор данных пользователя, выбор цели и активности, вычисление базального метаболизма по одной из доступных формул, расчет суточных энергозатрат и итоговую коррекцию под цель. Прототип интерфейса отражает основные функциональные блоки и позволяет наглядно представить взаимодействие пользователя с программой.

4.3 Разработка программного обеспечения

В данном разделе будут произведены разработки алгоритмов функционирования программы, разработан интерфейс программы, реализованы основные функции программы.

4.3.1 Разработка программного обеспечения

Программное обеспечение разработано в среде PyCharm на языке Python.

Архитектура приложения состоит из следующих модулей: основные вычисления содержатся в модуле algorithm.py, а графический интерфейс пользователя в модуле interface.py с использованием библиотеки PyQt5.

Главной функцией модуля с алгоритмами расчета является calculate_all. Она управляет всем процессом вычислений, последовательно вызывая вспомогательные функции. В сигнатуру calculate_all передаются антропометрические данные пользователя, коэффициент активности, выбранная цель и математическая формула, представлена на рисунке 12.

```
def calculate_all(  2 usages
    weight: float,
    height: float,
    age: int,
    gender: str,
    activity_factor: float,
    goal: str,
    formula: str = "mifflin",
    fat_percent: Optional[float] = None,
) -> Dict[str, Any]:
    bmr, formula_name = calculate_bmr(weight, height, age, gender, formula, fat_percent)
    tdee = calculate_tdee(bmr, activity_factor)
    target_calories = adjust_goal(tdee, goal)
    protein_g, fat_g, carbs_g, p_pct, f_pct, c_pct = calculate_macros(target_calories)

    return {
        "bmr": bmr,
        "tdee": tdee,
        "target_calories": target_calories,
        "protein_g": protein_g,
        "fat_g": fat_g,
        "carbs_g": carbs_g,
        "protein_pct": p_pct,
        "fat_pct": f_pct,
        "carbs_pct": c_pct,
        "formula_name": formula_name,
    }
```

Рисунок 12 – Функция calculate_all

Чтобы программа понимала, какую формулу применить, используется функция calculate_bmr. Она проверяет выбор пользователя и запускает один из четырех алгоритмов, представлена на рисунке 13.

```python
def calculate_bmr(  1 usage
    weight: float,
    height: float,
    age: int,
    gender: str,
    formula: str,
    fat_percent: Optional[float] = None,
) -> Tuple[float, str]:
    if formula == "mifflin": return mifflin(weight, height, age, gender), "Миффлин-Сан Жеор"
    if formula == "schofield": return schofield(weight, height, age, gender), "Шофилд (Детская)"
    if formula == "katch":
        if fat_percent is None: raise ValueError("Для Кетча-МакАрдла требуется процент жира")
        return katch_mcardle(weight, fat_percent), "Кетч-МакАрдл"
    if formula == "who": return who_formula(weight, age, gender), "Формула ВОЗ"
    raise ValueError("Unknown formula")
```

Рисунок 13 – Функция calculate_bmr

Функция mifflin – реализует формулу Миффлина – Сан Жеора, являющейся стандартной для большинства взрослых людей, представлена на рисунке 14.

```python
def mifflin_st_jeor(weight: float, height: float, age: int, gender: str) -> float:  1 usage
    if gender == "male":
        return (10 * weight) + (6.25 * height) - (5 * age) + 5
    return (10 * weight) + (6.25 * height) - (5 * age) - 161
```

Рисунок 14 – Функция mifflin

Функция schofield – реализует формулу Шофилда, предназначенную для детей. Алгоритм содержит внутреннее ветвление, применяющее различные коэффициенты для возрастных групп, а также выполняет конвертацию роста из сантиметров в метры, представлена на рисунке 15.

```python
def schofield(weight: float, height_cm: float, age: int, gender: str) -> float:  1 usage
    h_m = height_cm / 100.0
    if gender == "male":
        if age < 10:
            return (4.4 * weight) + (495 * h_m) + 499
        else:
            return (16.6 * weight) + (77 * h_m) + 572
    else:
        if age < 10:
            return (8.4 * weight) + (465 * h_m) + 200
        else:
            return (7.4 * weight) + (482 * h_m) + 217
```

Рисунок 15 – Функция schofield

Функция katch_mcardle – рассчитывает BMR на основе безжировой массы тела, вычитая процент подкожного жира из общей массы тела, представлена на рисунке 16.

```
def katch_mcardle(weight: float, fat_percent: float) -> float:  1 usage
    lbm = weight * (1 - (fat_percent / 100))
    return 370 + (21.6 * lbm)
```

Рисунок 16 – Функция katch_mcardle

Функция who_formula – реализует стандартизированный алгоритм Всемирной организации здравоохранения с ветвлением по возрастным диапазонам, представлена на рисунке 17.

```
def who_formula(weight: float, age: int, gender: str) -> float:  1 usage
    if gender == "male":
        if 18 <= age <= 30: return (15.3 * weight) + 679
        if 31 <= age <= 60: return (11.6 * weight) + 879
        return (13.5 * weight) + 487
    if 18 <= age <= 30: return (14.7 * weight) + 496
    if 31 <= age <= 60: return (8.7 * weight) + 829
    return (10.5 * weight) + 596
```

Рисунок 17 – Функция who_formula

Полученное значение BMR передается в функцию calculate_tdee, где оно умножается на коэффициент физической активности, представлена на рисунке 18.

```
def calculate_tdee(bmr: float, activity_factor: float) -> float:  1 usage
    return bmr * activity_factor
```

Рисунок 18 – Функция calculate_tdee

Функция adjust_goal изменяет итоговую калорийность. При выборе цели «Снижение веса» программа формирует дефицит в 500 ккал, а при выборе «Набор массы» – профицит в 500 ккал, представлена на рисунке 19.

```
def adjust_goal(tdee: float, goal: str) -> float:  1 usage
    if goal == "lose": return tdee - 500
    if goal == "gain": return tdee + 500
    return tdee
```

Рисунок 19 – Функция adjust_goal

Функция calculate_macros распределяет полученные калории по питательным веществам. Алгоритм использует классическую пропорцию: 30% – белки, 30% – жиры и 40% – углеводы. Для перевода калорий в граммы применяются коэффициенты Этуотера: целевая калорийность нутриента делится на 4 ккал/г для белков и углеводов, и на 9 ккал/г для жиров, представлена на рисунке 20.

```
def calculate_macros(calories: float) -> Tuple[float, float, float, int, int, int]:  1 usage
    p_pct, f_pct, c_pct = 30, 30, 40
    protein_g = (calories * (p_pct / 100)) / 4
    fat_g = (calories * (f_pct / 100)) / 9
    carbs_g = (calories * (c_pct / 100)) / 4
    return protein_g, fat_g, carbs_g, p_pct, f_pct, c_pct
```

Рисунок 20 – Функция calculate_macros

4.3.2 Разработка пользовательского интерфейса

Графический интерфейс пользователя разработан с использованием фреймворка PyQt5. Главное окно приложения «Калькулятор калорий» (класс CalorieCalculator) предназначено для ввода антропометрических данных, настройки параметров расчета и наглядной визуализации полученных результатов. Главное окно разделено на две основные логические зоны с помощью горизонтальной компоновки QHBoxLayout: панель ввода данных слева и панель вывода результатов справа. Главное окно приложения представлено на рисунке 21.

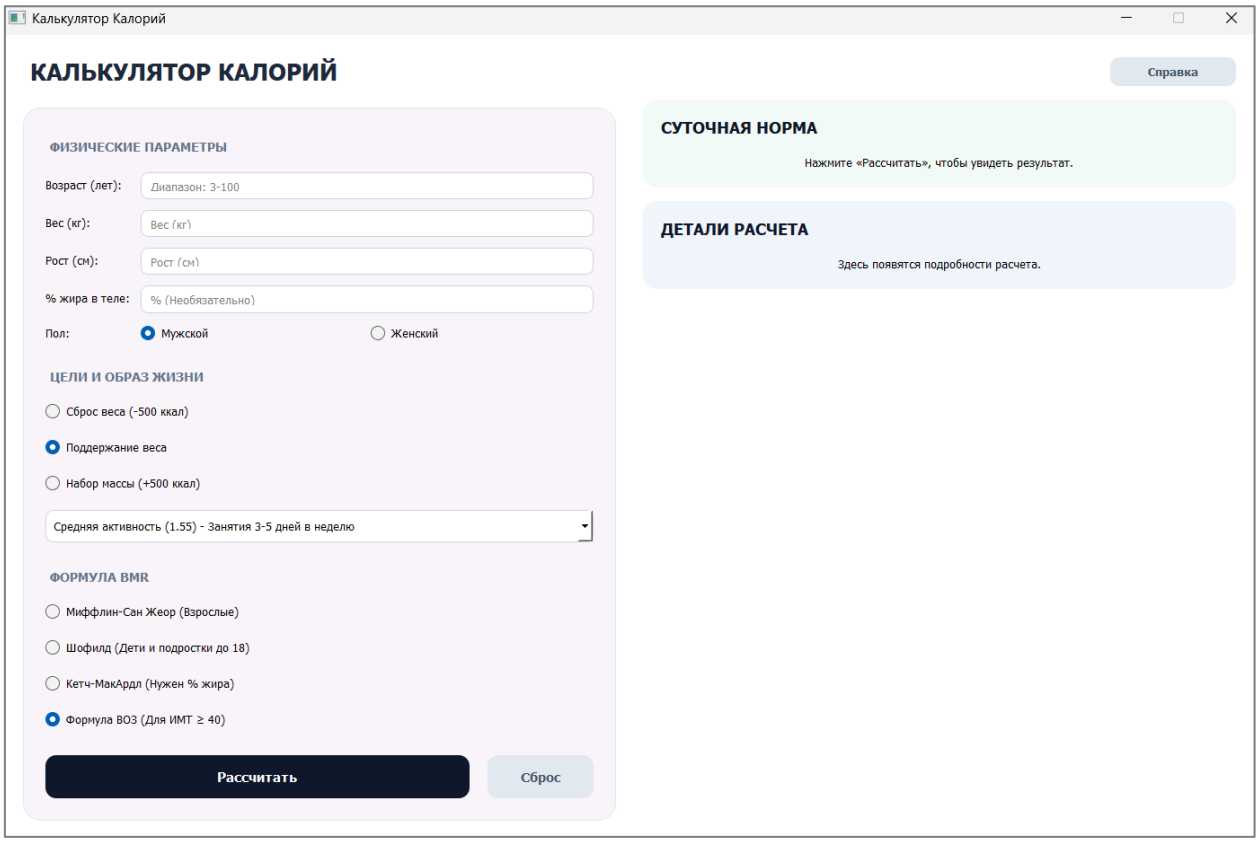


Рисунок 21 – Главное окно приложения

В левой части окна располагается форма ввода параметров, реализованная с помощью QFormLayout. Для реализации конкретных визуальных объектов используются следующие элементы библиотеки PyQt5.

Текстовые поля (QLineEdit) используются для ввода возраста, веса, роста и процента жира. Для защиты от ввода некорректных данных, например, букв вместо цифр, к этим полям привязаны валидаторы QIntValidator для целых чисел и QDoubleValidator для дробных чисел.

Важной особенностью интерфейса является интеллектуальная обработка вводимых данных. Для этого был реализован метод _age_editing_finished. Данная функция анализирует введенный возраст в режиме реального времени: если пользователь указывает возраст менее 18 лет, система автоматически переключает алгоритм расчета на формулу Шофилда, которая является приоритетной для детского организма. Это избавляет пользователя от необходимости самостоятельно изучать теоретическую базу, работа метода представлена на рисунке 22.

```python
def _age_editing_finished(self) -> None:  1 usage
    if not self.age_input.text().strip():
        return
    if not self.age_input.hasAcceptableInput():
        return

    age = int(self.age_input.text().strip())
    under_18 = age < 18

    if under_18 and not self.rb_schofield.isChecked():
        self.rb_schofield.setChecked(True)
        QMessageBox.information(
            self,
            "Рекомендация",
            "Для детей и подростков (до 18 лет) автоматически выбрана формула Шофилда (рекомендации ВОЗ).",
        )
```

Рисунка 22 – Метод _age_editing_finished

В дополнение к автоматизации выбора формул, в интерфейс встроена система обеспечения медицинской безопасности, реализованная через функцию _enforce_age_restrictions. В соответствии с рекомендациями ВОЗ, программа блокирует возможность выбора целей «Похудение» и «Набор массы» для несовершеннолетних пользователей. При обнаружении возраста младше 18 лет соответствующие радиокнопки становятся неактивными, принудительно устанавливая режим «Поддержание веса», что представлено на рисунке 23.

```python
def _enforce_age_restrictions(self) -> None:  4 usages
    age_text = self.age_input.text().strip()
    try:
        age = int(age_text) if age_text else None
    except ValueError:
        age = None

    under_18 = (age is not None and age < 18)
    if under_18 and not self.rb_schofield.isChecked():
        self.rb_schofield.setChecked(True)

    self.rb_lose.setEnabled(not under_18)
    self.rb_gain.setEnabled(not under_18)

    if under_18 and (self.rb_lose.isChecked() or self.rb_gain.isChecked()):
        self.rb_maintain.setChecked(True)
        QMessageBox.warning(
            self,
            "Ограничение по возрасту",
            "Для лиц младше 18 лет расчёт дефицита/профицита калорий не выполняется (рекомендации ВОЗ).\n"
            "Доступна только цель «Поддержание веса».",
        )

    if age is not None:
        w_min, w_max = self._get_weight_bounds_for_age(age)
        validator = self.weight_input.validator()
        if isinstance(validator, QDoubleValidator):
            validator.setBottom(float(w_min))
            validator.setTop(float(w_max))

    self._last_under_18 = under_18
```

Рисунка 23 – Функция _enforce_age_restrictions

Переключатели (QRadioButton) применяются для выбора пола, цели и формулы расчета. Для обеспечения взаимоисключающего выбора радиокнопки объединены в группы с помощью класса QButtonGroup. Выпадающий список (QComboBox) служит для выбора уровня физической активности пользователя.

Управление расчетами осуществляется с помощью кнопок управления (класс QPushButton). Кнопка «Рассчитать» инициирует метод calculate, который выполняет финальную валидацию всех полей. Если обязательные поля остаются пустыми или содержат некорректные значения, вызывается диалоговое окно ошибки (QMessageBox). После успешной проверки данные передаются в математический модуль.

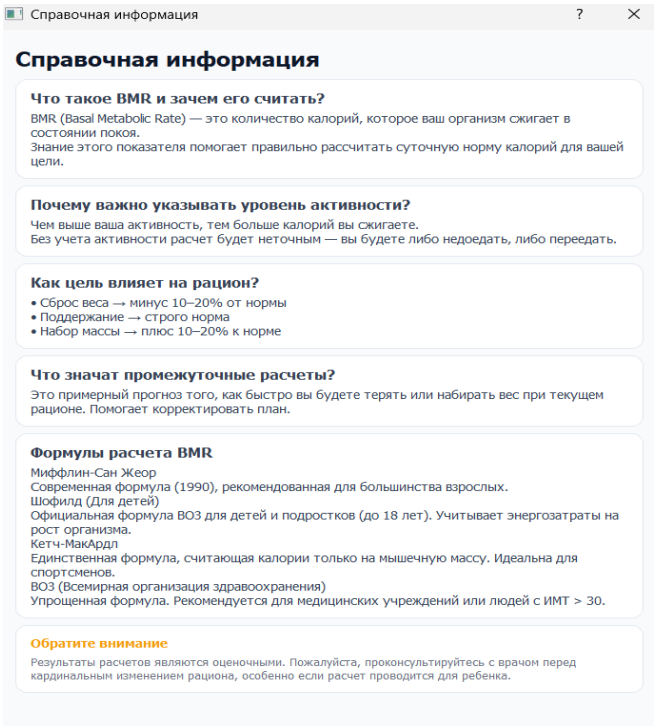
Для передачи вычисленных данных обратно в интерфейс используется метод _apply_results. Она принимает словарь с результатами и распределяет их по соответствующим виджетам правой панели, обновляя текстовые показатели и заполняя индикаторы макронутриентов, представлена на рисунке

24.

```python
def _apply_results(self, results: Dict[str, Any]) -> None:  1 usage
    self._last_results = results
    self.daily_calories_label.setText(f"{fmt_int(results['target_calories'])} ккал")

    self.block_protein.set_values(results["protein_pct"], results["protein_g"])
    self.block_fat.set_values(results["fat_pct"], results["fat_g"])
    self.block_carbs.set_values(results["carbs_pct"], results["carbs_g"])

    self.bmr_value_label.setText(f"{fmt_int(results['bmr'])} ккал/день")
    self.tdee_value_label.setText(f"{fmt_int(results['tdee'])} ккал/день")
    self.formula_value_label.setText(results["formula_name"])
    self.activity_value_label.setText(str(self._get_activity_factor()))

    self.target_placeholder.setVisible(False)
    self.target_results_widget.setVisible(True)
    self.calc_placeholder.setVisible(False)
    self.calc_results_widget.setVisible(True)
```

Рисунка 24 – Метод _apply_results

Кнопка «Сбросить» служит для возврата интерфейса в исходное состояние, вызывая метод clear для всех полей. Кнопка «Справка по формулам» вызывает дополнительное модальное окно (класс HelpDialog), которое содержит описание алгоритмов расчета. Окно справки представлено на рисунке 25.



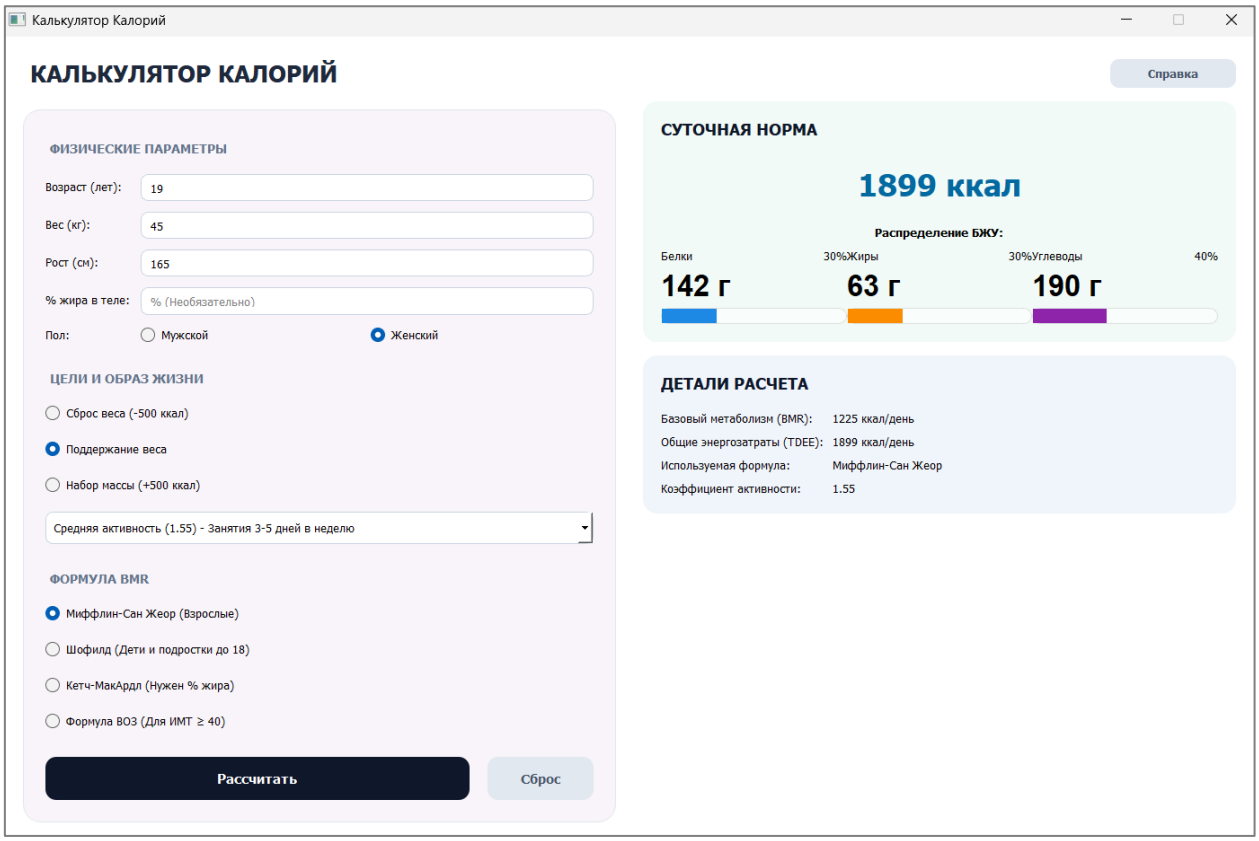
Рисунка 25 – Окно справочной информации

Для наглядного отображения рассчитанных макронутриентов в правой части окна был разработан класс MacroBlock. В его основе лежит использование индикаторов выполнения QProgressBar, которые визуализируют процентное соотношение нутриентов. Внутри класса реализован метод apply_progress_style, который с помощью каскадных таблиц стилей задает уникальное цветовое кодирование для белков, жиров и углеводов, а также настраивает радиус закругления элементов для соответствия современным стандартам дизайна, представлена на рисунке 26.

```
def apply_progress_style(self) -> None:  1 usage
    self.progress.setStyleSheet(
        f"""
        QProgressBar {{
            border: 1px solid rgba(0,0,0,0.1);
            border-radius: 9px;
            background: rgba(255,255,255,0.6);
        }}
        QProgressBar::chunk {{
            background-color: {self.visual.color};
            border-radius: 9px;
        }}
        """
    )
```

Рисунка 26 – Метод apply_progress_style

Точные значения в граммах выводятся с помощью текстовых меток QLabel. Внешний вид главного окна приложения с заполненными данными и результатами расчета представлен на рисунке 27.



Рисунка 27 – Внешний вид окна с заполненными данными

Разработанный пользовательский интерфейс калькулятора калорий отвечает требованиям интуитивности и функциональности. Использование современных графических компонентов PyQt5, системы автоматизации ввода и многоуровневой валидации данных позволяет минимизировать ошибки пользователя и обеспечивает высокую степень информационной безопасности. Разделение окна на две зоны обеспечивает удобство восприятия информации и мгновенную обратную связь системы на действия пользователя.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1) Команда Python и JetBrains провели ежегодный опрос питонистов [Электронный ресурс] // Skillbox Media. – URL: https://skillbox.ru/media/code/komanda-python-i-jetbrains-proveli-ezhegodnyy-opros-pitonistov/ (дата обращения: 31.01.2026).

2) Самые популярные языки программирования в 2024 году [Электронный ресурс] // Yandex Cloud Blog. – URL: https://yandex.cloud/ru/blog/most-popular-programming-languages (дата обращения: 31.01.2026).

3) SMITH, J. A., JOHNSON, M. L. Estimating Daily Energy Expenditure in Humans: A Review of Methods and Applications [Электронный ресурс] // LSU Digital Commons. – Baton Rouge: Louisiana State University, 2022. – URL: https://repository.lsu.edu/animalsciences_pubs/1560/ (дата обращения: 24.01.2026).

4) Основные принципы здорового питания и снижения веса для организации здоровой жизнедеятельности [Электронный ресурс] // CyberLeninka. – URL: https://cyberleninka.ru/article/n/osnovnye-printsipy-zdorovogo-pitaniya-i-snizheniya-vesa-dlya-organizatsii-zdorovoy-zhiznedeyatelnosti?ysclid=mks8ilpa6j993596113 (дата обращения: 24.01.2026).

5) Энергетический баланс: как управлять весом с помощью калорий [Электронный ресурс] // Яндекс Здоровье. – URL: https://yandex.ru/health/turbo/articles?id=7830 (дата обращения: 24.01.2026).

6) Суточные энергозатраты человека: физиологические механизмы и методы оценки общего расхода энергии [Электронный ресурс] / MIIN – Институт интегративного нутрициологии. – URL: https://miin.ru/blog/sutochnye-energozatraty-cheloveka-fiziologicheskie-mehanizmy-i-metody-oczenki-obshhego-rashoda-energi (дата обращения: 24.01.2026).

7) Вордстат – статистика поисковых запросов Яндекса [Электронный ресурс]: данные по фразе «калькулятор калорий». – URL: https://wordstat.yandex.ru/?region=all&view=table&words=%D0%BA%D0%B0%D0%BB%D1%8C%D0%BA%D1%83%D0%BB%D1%8F%D1%82%D0%BE%D1%80%20%D0%BA%D0%B0%D0%BB%D0%BE%D1%80%D0%B8%D0%B9 (дата обращения: 30.01.2026).

8) Вордстат – статистика поисковых запросов Яндекса [Электронный ресурс]: данные по фразе «сколько ккал». – URL: https://wordstat.yandex.ru/?region=all&view=table&words=%D1%81%D0%BA%D0%BE%D0%BB%D1%8C%D0%BA%D0%BE%20%D0%BA%D0%BA%D0%B0%D0%BB (дата обращения: 30.01.2026).

ПРИЛОЖЕНИЕ А

Модуль interface:

Ввымавчсро
Щл

```
from dataclasses import dataclass
from typing import Any, Dict, Optional
import sys
from pathlib import Path

from PyQt5.QtCore import Qt, QLocale
from PyQt5.QtGui import QDoubleValidator, QFont, QIntValidator, QValidator
from PyQt5.QtWidgets import (
    QButtonGroup,
    QComboBox,
    QFormLayout,
    QFrame,
    QLabel,
    QLineEdit,
    QMainWindow,
    QMessageBox,
    QPushButton,
    QProgressBar,
    QRadioButton,
    QVBoxLayout,
    QHBoxLayout,
    QWidget,
    QDialog
)

from algorithm import calculate_all

def _fmt_int(value: float) -> str:
    return str(int(round(value)))

def _fmt_activity_factor(factor: float) -> str:
    return f"{factor:g}"

@dataclass(frozen=True)
class MacroVisual:
    name: str
    color: str
class MacroBlock(QWidget):
    def __init__(self, visual: MacroVisual, parent: Optional[QWidget] = None):
        super().__init__(parent)
        self.visual = visual

        layout = QVBoxLayout(self)
        layout.setContentsMargins(0, 0, 0, 0)
        layout.setSpacing(6)

        top_row = QHBoxLayout()
        self.name_label = QLabel(self.visual.name)
        self.percent_label = QLabel("0%")
        self.percent_label.setAlignment(Qt.AlignRight | Qt.AlignVCenter)
        top_row.addWidget(self.name_label, 1)
        top_row.addWidget(self.percent_label, 0)
        self.percent_label.setVisible(False)

        self.grams_label = QLabel("0 г")
        self.grams_label.setFont(QFont("Arial", 20, QFont.Bold))
```

```
        self.progress = QProgressBar()
        self.progress.setRange(0, 100)
        self.progress.setValue(0)
        self.progress.setTextVisible(False)
        self.progress.setFixedHeight(18)

        self._apply_progress_style()

        layout.addLayout(top_row)
        layout.addWidget(self.grams_label)
        layout.addWidget(self.progress)

    def _apply_progress_style(self) -> None:
        self.progress.setStyleSheet(
            f"""
            QProgressBar {{
                border: 1px solid rgba(0,0,0,0.1);
                border-radius: 9px;
                background: rgba(255,255,255,0.6);
            }}
            QProgressBar::chunk {{
                background-color: {self.visual.color};
                border-radius: 9px;
            }}
            """
        )

    def set_values(self, percent: float, grams: float) -> None:
        pct_int = int(round(percent))
        grams_int = int(round(grams))
        self.percent_label.setText(f"{pct_int}%")
        self.grams_label.setText(f"{grams_int} г")
        self.progress.setValue(pct_int)


class HelpDialog(QDialog):
    def __init__(self, parent=None):
        super().__init__(parent)
        self.setWindowTitle("Справочная информация")
        self.setMinimumSize(700, 850)
        self.setStyleSheet("QDialog { background-color: #f8fafc; font-family: 'Segoe UI', Arial; }")

        main_layout = QVBoxLayout(self)
        main_layout.setContentsMargins(0, 0, 0, 0)
        main_layout.setSpacing(0)

        header = QWidget()
        header.setStyleSheet("background-color: transparent;")
        header_layout = QVBoxLayout(header)
        header_layout.setContentsMargins(20, 20, 20, 10)

        title = QLabel("Справочная информация")
        title.setStyleSheet("font-size: 24px; font-weight: 800; color: #0f172a;")

        header_layout.addWidget(title)
        main_layout.addWidget(header)

        content_widget = QWidget()
        content_widget.setStyleSheet("background-color: transparent;")
        self.content_layout = QVBoxLayout(content_widget)
        self.content_layout.setContentsMargins(20, 0, 20, 20)
        self.content_layout.setSpacing(8)
```

```
        self._populate_cards()
        self.content_layout.addStretch()

        main_layout.addWidget(content_widget)

    def _populate_cards(self):
        content_blocks = [
            ("Что такое BMR и TDEE?",
             "BMR – базовая энергия для жизни (дыхание, работа сердца, обмен веществ).\n\n"
             "TDEE – ваши полные энергозатраты за сутки (BMR × активность)."),
            ("Уровень активности",
             "Множитель, учитывающий движение. Без него расчет будет неверным: вы либо не доберете
энергии, либо получите избыток."),
            ("Влияние цели на рацион (Дефицит и Профицит)",
             "Для изменения веса мы корректируем норму TDEE:\n\n"
             "Сброс веса: дефицит калорий от нормы TDEE.\n\n"
             "Набор массы: профицит калорий к норме TDEE.\n\n"
             "Поддержание: потребление строго по норме TDEE."),
            ("Почему ±500 ккал?",
             "Это «золотой стандарт» диетологии. Дефицит в 500 ккал позволяет сжигать примерно 0.5 кг жира в
неделю – "
             "это считается самой безопасной и стабильной скоростью похудения для взрослого человека без
вреда для здоровья."),
            ("Формулы расчета BMR",
             "Миффлин-Сан Жеор: современный стандарт для взрослых (с 1990 г.).\n\n"
             "Шофилд: официальная формула ВОЗ для детей и подростков до 18 лет.\n\n"
             "Кетч-МакАрдл: расчет по сухой мышечной массе (для атлетов).\n\n"
             "ВОЗ (Всемирная орг. здравоохранения): универсальная медицинская база."),
            ("Обратите внимание",
             "Результаты являются оценочными. Перед кардинальным изменением рациона, особенно для
ребенка, проконсультируйтесь с врачом.")
        ]

        for title, text in content_blocks:
            card = QFrame()
            card.setStyleSheet("""
                QFrame {
                    background-color: white;
                    border: 1px solid #e2e8f0;
                    border-radius: 12px;
                }
            """)
            card_layout = QVBoxLayout(card)
            card_layout.setContentsMargins(15, 12, 15, 12)
            card_layout.setSpacing(6)

            lbl_title = QLabel(title)
            lbl_title.setWordWrap(True)
            lbl_title.setStyleSheet("font-size: 16px; font-weight: 700; color: #364153; border: none;")

            lbl_text = QLabel(text)
            lbl_text.setWordWrap(True)
            lbl_text.setStyleSheet("font-size: 14px; color: #334155; line-height: 1.4; border: none;")

            if title == "Обратите внимание":
                lbl_title.setStyleSheet("font-size: 14px; font-weight: 700; color: #f59e0b; border: none;")
                lbl_text.setStyleSheet("font-size: 13px; color: #6b7280; border: none;")

            card_layout.addWidget(lbl_title)
            card_layout.addWidget(lbl_text)
            self.content_layout.addWidget(card)
```

```
class CalorieCalculator(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Калькулятор Калорий")
        self.setFixedSize(1400, 900)

        self._last_results: Optional[Dict[str, Any]] = None
        self._last_under_18: Optional[bool] = None

        self._build_ui()
        self._apply_styles()
        self._wire_events()

    def _build_ui(self) -> None:
        central = QWidget(self)
        self.setCentralWidget(central)

        root_layout = QHBoxLayout(central)
        root_layout.setContentsMargins(25, 25, 25, 25)
        root_layout.setSpacing(30)

        self.left_panel = QWidget()
        self.left_panel.setObjectName("leftPanel")
        self.left_layout = QVBoxLayout(self.left_panel)
        self.left_layout.setContentsMargins(0, 0, 0, 0)
        self.left_layout.setSpacing(20)

        self.left_title = QLabel("КАЛЬКУЛЯТОР КАЛОРИЙ")
        self.left_title.setObjectName("leftMainTitle")

        self.input_container = QFrame()
        self.input_container.setObjectName("inputContainer")
        self._build_consolidated_inputs()

        self.left_layout.addWidget(self.left_title)
        self.left_layout.addWidget(self.input_container)
        self.left_layout.addStretch(1)

        self.right_panel = QWidget()
        self.right_panel.setObjectName("rightPanel")
        self.right_layout = QVBoxLayout(self.right_panel)
        self.right_layout.setContentsMargins(0, 0, 0, 0)
        self.right_layout.setSpacing(15)

        self.right_top_row = QHBoxLayout()
        self.right_top_row.addStretch(1)
        self.btn_help = QPushButton("Справка")
        self.btn_help.setObjectName("helpButton")
        self.btn_help.setFixedSize(140, 32)
        self.right_top_row.addWidget(self.btn_help)

        self.group_target = QFrame()
        self.group_target.setObjectName("targetCard")
        self._build_target_card()

        self.group_intermediate = QFrame()
        self.group_intermediate.setObjectName("detailsCard")
        self._build_details_card()

        self.right_layout.addLayout(self.right_top_row)
        self.right_layout.addWidget(self.group_target)
        self.right_layout.addWidget(self.group_intermediate)
```

```
        self.right_layout.addStretch(1)

        root_layout.addWidget(self.left_panel, 1)
        root_layout.addWidget(self.right_panel, 1)

    def _build_consolidated_inputs(self) -> None:
        layout = QVBoxLayout(self.input_container)
        layout.setContentsMargins(24, 24, 24, 24)
        layout.setSpacing(20)

        specs_title = QLabel("Физические параметры")
        specs_title.setObjectName("sectionTitle")
        layout.addWidget(specs_title)

        form_layout = QFormLayout()
        form_layout.setSpacing(12)

        self.age_input = QLineEdit()
        self.age_input.setPlaceholderText("Диапазон: 3-100")
        self.age_input.setValidator(QIntValidator(3, 100, self))

        self.weight_input = QLineEdit()
        self.weight_input.setPlaceholderText("Вес (кг)")
        self.weight_input.setValidator(QDoubleValidator(35.0, 250.0, 2, self))

        self.height_input = QLineEdit()
        self.height_input.setPlaceholderText("Рост (см)")
        self.height_input.setValidator(QDoubleValidator(80.0, 250.0, 1, self))

        self.fat_input = QLineEdit()
        self.fat_input.setPlaceholderText("Обязательно для Кетча-МакАрдла")
        self.fat_input.setValidator(QDoubleValidator(5.0, 50.0, 2, self))

        form_layout.addRow("Возраст (лет):", self.age_input)
        form_layout.addRow("Вес (кг):", self.weight_input)
        form_layout.addRow("Рост (см):", self.height_input)
        form_layout.addRow("% жира в теле:", self.fat_input)

        gender_widget = QWidget()
        gender_layout = QHBoxLayout(gender_widget)
        gender_layout.setContentsMargins(0, 0, 0, 0)
        self.rb_male = QRadioButton("Мужской")
        self.rb_female = QRadioButton("Женский")
        self.rb_male.setChecked(True)
        gender_layout.addWidget(self.rb_male)
        gender_layout.addWidget(self.rb_female)
        self.gender_group = QButtonGroup(self)
        self.gender_group.addButton(self.rb_male)
        self.gender_group.addButton(self.rb_female)
        form_layout.addRow("Пол:", gender_widget)

        layout.addLayout(form_layout)

        goal_title = QLabel("Цели и образ жизни")
        goal_title.setObjectName("sectionTitle")
        layout.addWidget(goal_title)

        self.rb_lose = QRadioButton("Сброс веса (-500 ккал)")
        self.rb_maintain = QRadioButton("Поддержание веса")
        self.rb_gain = QRadioButton("Набор массы (+500 ккал)")
        self.rb_maintain.setChecked(True)
        self.goal_group = QButtonGroup(self)
        for rb in (self.rb_lose, self.rb_maintain, self.rb_gain):
```

```
        self.goal_group.addButton(rb)
        layout.addWidget(rb)

    self.activity_combo = QComboBox()
    self.activity_combo.addItems([
        "Сидячий (1.2) - Без нагрузок",
        "Низкая активность (1.375) - Занятия 1-3 дня в неделю",
        "Средняя активность (1.55) - Занятия 3-5 дней в неделю",
        "Высокая активность (1.725) - Занятия 6-7 дней в неделю",
        "Экстремальная активность (1.9) - Тяжелая работа / 2 тренировки в день",
    ])
    self.activity_combo.setCurrentIndex(2)
    layout.addWidget(self.activity_combo)

    method_title = QLabel("Формула BMR")
    method_title.setObjectName("sectionTitle")
    layout.addWidget(method_title)

    self.rb_mifflin = QRadioButton("Миффлин-Сан Жеор (Взрослые)")
    self.rb_schofield = QRadioButton("Шофилд (Дети и подростки до 18)")
    self.rb_katch = QRadioButton("Кетч-МакАрдл (Нужен % жира)")
    self.rb_who = QRadioButton("Формула ВОЗ (медицинская, для людей с ожирением)")
    self.rb_mifflin.setChecked(True)
    self.formula_group = QButtonGroup(self)
    for rb in (self.rb_mifflin, self.rb_schofield, self.rb_katch, self.rb_who):
        self.formula_group.addButton(rb)
        layout.addWidget(rb)

    layout.addSpacing(10)
    self.buttons_row = QHBoxLayout()
    self.btn_calc = QPushButton("Рассчитать")
    self.btn_calc.setObjectName("calcButton")
    self.btn_reset = QPushButton("Сброс")
    self.btn_reset.setObjectName("resetButton")

    self.buttons_row.addWidget(self.btn_calc, 4)
    self.buttons_row.addWidget(self.btn_reset, 1)
    layout.addLayout(self.buttons_row)

def _build_target_card(self) -> None:
    layout = QVBoxLayout(self.group_target)
    layout.setContentsMargins(20, 20, 20, 20)
    layout.setSpacing(10)

    title = QLabel("СУТОЧНАЯ НОРМА")
    title.setObjectName("cardHeader")
    layout.addWidget(title)

    layout.addSpacing(10)

    self.target_placeholder = QLabel("Нажмите «Рассчитать», чтобы увидеть результат.")
    self.target_placeholder.setAlignment(Qt.AlignCenter)
    layout.addWidget(self.target_placeholder)

    self.target_results_widget = QWidget()
    self.target_results_widget.setObjectName("targetResultsWidget")
    res_layout = QVBoxLayout(self.target_results_widget)
    res_layout.setContentsMargins(0, 0, 0, 0)
    res_layout.setSpacing(0)

    self.daily_calories_label = QLabel("– ккал")
    self.daily_calories_label.setObjectName("bigKcal")
    self.daily_calories_label.setAlignment(Qt.AlignCenter)
```

```
    macro_row = QHBoxLayout()
    macro_row.setContentsMargins(0, 0, 0, 0)
    self.block_protein = MacroBlock(MacroVisual("Белки", "#1E88E5"))
    self.block_fat = MacroBlock(MacroVisual("Жиры", "#FB8C00"))
    self.block_carbs = MacroBlock(MacroVisual("Углеводы", "#8E24AA"))
    macro_row.addWidget(self.block_protein)
    macro_row.addWidget(self.block_fat)
    macro_row.addWidget(self.block_carbs)

    res_layout.addWidget(self.daily_calories_label)
    res_layout.addSpacing(15)
    self.macro_title_label = QLabel("Распределение БЖУ:")
    self.macro_title_label.setObjectName("macroTitle")
    self.macro_title_label.setAlignment(Qt.AlignLeft | Qt.AlignVCenter)
    res_layout.addWidget(self.macro_title_label)
    res_layout.addSpacing(10)
    res_layout.addLayout(macro_row)

    layout.addWidget(self.target_results_widget)
    self.target_results_widget.setVisible(False)

def _build_details_card(self) -> None:
    layout = QVBoxLayout(self.group_intermediate)
    layout.setContentsMargins(20, 20, 20, 20)
    layout.setSpacing(10)

    title = QLabel("ДЕТАЛИ РАСЧЕТА")
    title.setObjectName("cardHeader")
    layout.addWidget(title)

    layout.addSpacing(10)

    self.calc_placeholder = QLabel("Здесь появятся подробности расчета.")
    self.calc_placeholder.setAlignment(Qt.AlignCenter)
    layout.addWidget(self.calc_placeholder)

    self.calc_results_widget = QWidget()
    det_layout = QFormLayout(self.calc_results_widget)
    det_layout.setContentsMargins(0, 0, 0, 0)
    det_layout.setSpacing(10)

    self.bmr_value_label = QLabel("–")
    self.tdee_value_label = QLabel("–")
    self.formula_value_label = QLabel("–")
    self.activity_value_label = QLabel("–")

    det_layout.addRow("Базовый метаболизм (BMR):", self.bmr_value_label)
    det_layout.addRow("Общие энергозатраты (TDEE):", self.tdee_value_label)
    det_layout.addRow("Используемая формула:", self.formula_value_label)
    det_layout.addRow("Коэффициент активности:", self.activity_value_label)

    layout.addWidget(self.calc_results_widget)
    self.calc_results_widget.setVisible(False)

def _apply_styles(self) -> None:
    self.setStyleSheet("""
        QMainWindow { background-color: #FFFFFF; font-family: 'Segoe UI', Arial; }

        QLabel#leftMainTitle {
            font-size: 26px; font-weight: 800; color: #1e293b;
            margin-bottom: 5px;
        }
```

```
        QFrame#inputContainer {
            background-color: #f8f4f9;
            border: 1px solid #e2e8f0;
            border-radius: 20px;
        }

        QLabel#sectionTitle {
            font-size: 14px; font-weight: 800; color: #1e293b;
            margin-top: 10px;
        }

        QFrame#targetCard {
            background-color: #f1faf6;
            border-radius: 16px;
        }
        QFrame#detailsCard {
            background-color: #f0f5fb;
            border-radius: 16px;
        }

        QWidget#targetResultsWidget { border: none; background: transparent;}

        QLabel#cardHeader { font-size: 18px; font-weight: 800; color: #0f172a; border: none; background:
transparent;}
        QLabel#bigKcal { font-size: 34px; font-weight: 900; color: #0f172a; margin: 10px 0; border: none;
background: transparent;}
        QLabel#macroTitle { font-size: 15px; font-weight: 700; color: #0f172a; border: none; background:
transparent;}
        QLabel { background: transparent; }

        QLineEdit, QComboBox {
            border: 1px solid #cbd5e1; border-radius: 8px; padding: 8px; background: white;
        }
        QLineEdit:focus { border: 2px solid #3b82f6; }

        QPushButton#calcButton {
            background-color: #0f172a; color: white; border-radius: 12px;
            font-weight: 700; font-size: 15px; height: 48px;
        }
        QPushButton#calcButton:hover { background-color: #1e293b; }

        QPushButton#resetButton {
            background-color: #e2e8f0; color: #475569; border-radius: 12px;
            font-weight: 600; font-size: 14px; height: 48px;
        }
        QPushButton#resetButton:hover { background-color: #cbd5e1; }

        QPushButton#helpButton {
            background-color: #e2e8f0; border: none; border-radius: 10px;
            color: #475569; font-weight: 600;
        }
        QPushButton#helpButton:hover {
            background-color: #cbd5e1;
        }
    """)

    def _wire_events(self) -> None:
        self.btn_calc.clicked.connect(self.calculate)
        self.btn_reset.clicked.connect(self.reset_fields)
        self.btn_help.clicked.connect(self.show_help_from_file)

        self.rb_mifflin.clicked.connect(self._check_formula_age_warning)
```

```
        self.rb_schofield.clicked.connect(self._check_formula_age_warning)
        self.rb_who.clicked.connect(self._check_formula_age_warning)
        self.rb_katch.clicked.connect(self._check_formula_age_warning)

        self.age_input.textChanged.connect(self._enforce_age_restrictions)
        self.age_input.editingFinished.connect(self._age_editing_finished)
        self.rb_lose.toggled.connect(self._enforce_age_restrictions)
        self.rb_gain.toggled.connect(self._enforce_age_restrictions)
        self._enforce_age_restrictions()

    def _check_formula_age_warning(self):
        age_text = self.age_input.text()
        if not age_text:
            return

        if not self.age_input.hasAcceptableInput():
            return

        try:
            age = int(age_text)
        except ValueError:
            return

        if self.rb_schofield.isChecked() and age >= 18:
            QMessageBox.information(self, "Внимание",
                                    "Формула Шофилда предназначена для детей и подростков (до 18 лет).\n\nДля
взрослых рекомендуется Миффлин-Сан Жеор.")
        elif not self.rb_schofield.isChecked() and age < 18:
            QMessageBox.information(self, "Внимание",
                                    "Для детей и подростков (до 18 лет) рекомендуется использовать формулу
Шофилда.\n\nВзрослые формулы могут дать неверный результат.")

    def _enforce_age_restrictions(self) -> None:
        age_text = self.age_input.text().strip()
        try:
            age = int(age_text) if age_text else None
        except ValueError:
            age = None

        under_18 = (age is not None and age < 18)
        if under_18 and not self.rb_schofield.isChecked():
            self.rb_schofield.setChecked(True)

        self.rb_lose.setEnabled(not under_18)
        self.rb_gain.setEnabled(not under_18)

        if under_18 and (self.rb_lose.isChecked() or self.rb_gain.isChecked()):
            self.rb_maintain.setChecked(True)
            QMessageBox.warning(
                self,
                "Ограничение по возрасту",
                "Для лиц младше 18 лет расчёт дефицита/профицита калорий не выполняется (рекомендации
ВОЗ).\n"
                "Доступна только цель «Поддержание веса».",
            )

        if age is not None:
            w_min, w_max = self._get_weight_bounds_for_age(age)
            validator = self.weight_input.validator()
            if isinstance(validator, QDoubleValidator):
                validator.setBottom(float(w_min))
                validator.setTop(float(w_max))
```

```
        self._last_under_18 = under_18

    def _age_editing_finished(self) -> None:
        if not self.age_input.text().strip():
            return
        if not self.age_input.hasAcceptableInput():
            return

        age = int(self.age_input.text().strip())
        under_18 = age < 18

        if under_18 and not self.rb_schofield.isChecked():
            self.rb_schofield.setChecked(True)
            QMessageBox.information(
                self,
                "Рекомендация",
                "Для детей и подростков (до 18 лет) автоматически выбрана формула Шофилда (рекомендации
ВОЗ).",
            )

    def _get_weight_bounds_for_age(self, age: int) -> tuple[float, float]:
        if age < 10:
            return 3.0, 60.0
        if age < 18:
            return 25.0, 150.0
        return 35.0, 250.0

    def _get_activity_factor(self) -> float:
        text = self.activity_combo.currentText()
        try:
            return float(text[text.find("(") + 1: text.find(")")])
        except:
            return 1.55

    def _get_goal_code(self) -> str:
        if self.rb_lose.isChecked(): return "lose"
        if self.rb_gain.isChecked(): return "gain"
        return "maintain"

    def _get_formula_code(self) -> str:
        if self.rb_mifflin.isChecked(): return "mifflin"
        if self.rb_schofield.isChecked(): return "schofield"
        if self.rb_katch.isChecked(): return "katch"
        return "who"

    def calculate(self) -> None:
        try:
            age_text = self.age_input.text().strip()
            weight_text_raw = self.weight_input.text().strip()
            height_text_raw = self.height_input.text().strip()
            fat_text_raw = self.fat_input.text().strip()

            if not age_text:
                raise ValueError("Поле «Возраст» обязательно для заполнения.")
            if not self.age_input.hasAcceptableInput():
                raise ValueError("Некорректный возраст. Введите целое число в диапазоне 3–100.")
            age = int(age_text)

            if not weight_text_raw:
                raise ValueError("Поле «Вес» обязательно для заполнения.")
            if not self.weight_input.hasAcceptableInput():
                w_min, w_max = self._get_weight_bounds_for_age(age)
```

```
            raise ValueError(f"Некорректный вес. Введите число в диапазоне {w_min:g}–{w_max:g} кг для
указанного возраста.")
        weight = float(weight_text_raw.replace(",", "."))

        if not height_text_raw:
            raise ValueError("Поле «Рост» обязательно для заполнения.")
        if not self.height_input.hasAcceptableInput():
            raise ValueError("Некорректный рост. Введите число в диапазоне 80–250 см.")
        height = float(height_text_raw.replace(",", "."))

        fat: Optional[float]
        if fat_text_raw:
            if not self.fat_input.hasAcceptableInput():
                raise ValueError("Некорректный процент жира. Введите число в диапазоне 5–50%.")
            fat = float(fat_text_raw.replace(",", "."))
        else:
            fat = None

        if age < 18 and self._get_goal_code() != "maintain":
            self.rb_maintain.setChecked(True)
            raise ValueError(
                "Для лиц младше 18 лет расчёт дефицита/профицита калорий не выполняется (рекомендации
ВОЗ). "
                "Выберите цель «Поддержание веса»."
            )

        f_code = self._get_formula_code()

        if age < 18 and f_code != "schofield":
            reply = QMessageBox.warning(self, "Подтверждение",
                                        "Вы используете взрослую формулу для ребенка. Это может исказить норму
питания. Продолжить?",
                                        QMessageBox.Yes | QMessageBox.No)
            if reply == QMessageBox.No: return

        if age >= 18 and f_code == "schofield":
            reply = QMessageBox.warning(self, "Подтверждение",
                                        "Вы используете детскую формулу для взрослого. Продолжить?",
                                        QMessageBox.Yes | QMessageBox.No)
            if reply == QMessageBox.No: return

        if f_code == "katch" and fat is None:
            raise ValueError("Для формулы Кетча-МакАрдла требуется указать процент жира.")

        results = calculate_all(
            weight=weight, height=height, age=age,
            gender="male" if self.rb_male.isChecked() else "female",
            activity_factor=self._get_activity_factor(),
            goal=self._get_goal_code(),
            formula=f_code, fat_percent=fat
        )

        self._apply_results_to_ui(results)

    except ValueError as e:
        QMessageBox.warning(self, "Ошибка ввода", str(e))
    except Exception as e:
        QMessageBox.critical(self, "Ошибка", f"Произошла непредвиденная ошибка: {e}")

def _apply_results_to_ui(self, results: Dict[str, Any]) -> None:
    self._last_results = results
    self.daily_calories_label.setText(f"{_fmt_int(results['target_calories'])} ккал")
```

```
        self.block_protein.set_values(results["protein_pct"], results["protein_g"])
        self.block_fat.set_values(results["fat_pct"], results["fat_g"])
        self.block_carbs.set_values(results["carbs_pct"], results["carbs_g"])

        self.bmr_value_label.setText(f"{_fmt_int(results['bmr'])} ккал/день")
        self.tdee_value_label.setText(f"{_fmt_int(results['tdee'])} ккал/день")
        self.formula_value_label.setText(results["formula_name"])
        self.activity_value_label.setText(str(self._get_activity_factor()))

        self.target_placeholder.setVisible(False)
        self.target_results_widget.setVisible(True)
        self.calc_placeholder.setVisible(False)
        self.calc_results_widget.setVisible(True)

    def reset_fields(self) -> None:
        for widget in [self.age_input, self.weight_input, self.height_input, self.fat_input]:
            widget.clear()
        self.rb_maintain.setChecked(True)
        self.rb_mifflin.setChecked(True)
        self.activity_combo.setCurrentIndex(2)

        self.target_results_widget.setVisible(False)
        self.target_placeholder.setVisible(True)
        self.calc_results_widget.setVisible(False)
        self.calc_placeholder.setVisible(True)

    def show_help_from_file(self) -> None:
        dialog = HelpDialog(self)
        dialog.exec_()
```

Модуль algorithm.py

```
from __future__ import annotations
from typing import Optional, Dict, Any, Tuple

def mifflin(weight: float, height: float, age: int, gender: str) -> float:
    if gender == "male":
        return (10 * weight) + (6.25 * height) - (5 * age) + 5
    return (10 * weight) + (6.25 * height) - (5 * age) - 161

def schofield(weight: float, height_cm: float, age: int, gender: str) -> float:
    h_m = height_cm / 100.0
    if gender == "male":
        if age < 10:
            return (4.4 * weight) + (495 * h_m) + 499
        else:
            return (16.6 * weight) + (77 * h_m) + 572
    else:
        if age < 10:
            return (8.4 * weight) + (465 * h_m) + 200
        else:
            return (7.4 * weight) + (482 * h_m) + 217


def katch_mcardle(weight: float, fat_percent: float) -> float:
    lbm = weight * (1 - (fat_percent / 100))
    return 370 + (21.6 * lbm)


def who_formula(weight: float, age: int, gender: str) -> float:
```

```
    if gender == "male":
        if age < 18: return (17.5 * weight) + 651
        if age < 30: return (15.3 * weight) + 679
        if age < 60: return (11.6 * weight) + 879
        return (13.5 * weight) + 487
    else:
        if age < 18: return (12.2 * weight) + 746
        if age < 30: return (14.7 * weight) + 496
        if age < 60: return (8.7 * weight) + 829
        return (10.5 * weight) + 596


def calculate_tdee(bmr: float, activity_factor: float) -> float:
    return bmr * activity_factor


def adjust_goal(tdee: float, goal: str) -> float:
    if goal == "lose": return tdee - 500
    if goal == "gain": return tdee + 500
    return tdee


def calculate_macros(calories: float) -> Tuple[float, float, float, int, int, int]:
    # Стандарт: 30% белки, 30% жиры, 40% углеводы
    p_cal = calories * 0.30
    f_cal = calories * 0.30
    c_cal = calories * 0.40
    return p_cal / 4, f_cal / 9, c_cal / 4, 30, 30, 40


def calculate_bmr(
    weight: float,
    height: float,
    age: int,
    gender: str,
    formula: str,
    fat_percent: Optional[float] = None,
) -> Tuple[float, str]:
    if formula == "mifflin": return mifflin(weight, height, age, gender), "Миффлин-Сан Жеор"
    if formula == "schofield": return schofield(weight, height, age, gender), "Шофилд (Детская)"
    if formula == "katch":
        if fat_percent is None: raise ValueError("Для Кетча-МакАрдла требуется процент жира")
        return katch_mcardle(weight, fat_percent), "Кетч-МакАрдл"
    if formula == "who": return who_formula(weight, age, gender), "Формула ВОЗ"
    raise ValueError("Unknown formula")


def calculate_all(
    weight: float,
    height: float,
    age: int,
    gender: str,
    activity_factor: float,
    goal: str,
    formula: str = "mifflin",
    fat_percent: Optional[float] = None,
) -> Dict[str, Any]:
```

```
    if not isinstance(weight, (int, float)) or not isinstance(height, (int, float)):
        raise TypeError("Вес и рост должны быть числовыми значениями")
    if not isinstance(age, int):
        raise TypeError("Возраст должен быть целым числом")

    bmr, formula_name = calculate_bmr(weight, height, age, gender, formula, fat_percent)
    tdee = calculate_tdee(bmr, activity_factor)
    target_calories = adjust_goal(tdee, goal)
    protein_g, fat_g, carbs_g, p_pct, f_pct, c_pct = calculate_macros(target_calories)

    return {
        "bmr": bmr,
        "tdee": tdee,
        "target_calories": target_calories,
        "protein_g": protein_g,
        "fat_g": fat_g,
        "carbs_g": carbs_g,
        "protein_pct": p_pct,
        "fat_pct": f_pct,
        "carbs_pct": c_pct,
        "formula_name": formula_name,
    }
```

Модуль main.py

```
import sys
from pathlib import Path

try:
    import PyQt5
except ModuleNotFoundError:
    project_root = Path(__file__).resolve().parent
    site_packages = project_root / ".venv" / "Lib" / "site-packages"
    if site_packages.exists():
        sys.path.insert(0, str(site_packages))

from PyQt5.QtWidgets import QApplication

from interface import CalorieCalculator

if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = CalorieCalculator()
    window.show()
    sys.exit(app.exec_())
```